

Архітектура персистентної пам'яті для довгострокових автономних місій з ротацією операторів: дворежимне витягування та сигнал корекції

Овчаров В. О.*
LEX AI LLC, Київ, Україна

2026

Анотація

Автономні агентні системи, що виконують довгострокові місії (години — тижні), потребують підсистеми пам'яті, одиницею витягування якої є оперативне рішення, а не діалоговий обмін або фрагмент коду. Існуючі системи пам'яті для великих мовних моделей (MemGPT, Mem0, generative agents) організовані навколо діалогових епізодів і не підтримують ключову вимогу реальних місій — безперервність управління при ротації операторів.

Ми пропонуємо трирівневу декомпозицію пам'яті (предметна область / робочий процес / оператор) з дворежимним витягуванням: pull-режим для активних сесій та push-режим для фонових оновлень контексту простоюючих задач. Третій внесок — *сигнал корекції витягування* (retrieval-correction signal): ситуації, коли корекція оператора була б непотрібною, якби система вчасно подала релевантний контекст. Цей сигнал масштабується з автономністю агента і доповнює традиційні метрики нагляду.

Архітектура верифікована на платформі з 70+ інструментами та 380M+ записів; базові вимірювання на 304 сесіях підтверджують медіанну вартість ініціалізації 30 115 токенів з 60% надлишковістю контексту. Обговорюється застосовність запропонованої архітектури до систем управління безпілотними літальними апаратами та ситуаційних центрів, де ротація операторів є штатною процедурою.

Ключові слова: персистентна пам'ять, автономні агентні системи, ротація операторів, дворежимне витягування, сигнал корекції, БПЛА, онтологічно-керовані системи

1 Вступ: проблема безперервності управління

1.1 Актуальність

Автономні агентні системи — від великих мовних моделей (LLM), що працюють як програмні агенти, до безпілотних літальних апаратів (БПЛА) та наземних роботизованих комплексів — дедалі частіше виконують місії тривалістю від годин до тижнів. У таких системах людина-оператор здійснює нагляд (oversight) за автономним агентом: коригує рішення, змінює пріоритети, підтверджує або відхиляє дії.

*Кореспонденція: volodymyr@legal.org.ua

При ротації операторів (зміна екіпажу БПЛА, зміна вахти ситуативного центру, передача управління між змінами) виникає критична проблема: **втрата контексту рішень**. Новий оператор повинен знати:

- які рішення прийнято попередником і чому;
- які корекції внесено та на підставі якої інформації;
- які загрози виявлено, але не усунено;
- який контекст місії залишається актуальним.

Існуючі підходи до пам'яті агентних систем [1, 11, 19] організовані навколо діалогових епізодів і не вирішують проблему зміни операторів. Системи доповненого витягування для програмних агентів [24] витягують фрагменти без провенансу рішень. Моделі з довгим контекстом [20] демонструють деградацію уваги після $\sim 200K$ токенів [8].

1.2 Зв'язок зі школою кібернетичного управління

Запропонована робота розвиває принципи школи онтологічно-керованих систем, закладені в [13]: формальна онтологічна структура повинна не лише описувати систему, а *контролювати* її поведінку. Цей принцип послідовно застосовувався на дедалі вищих рівнях обчислювального стеку — від архітектури систем [13] через обробку природномовних текстів [14, 15] до контролю виходу великих мовних моделей [16, 17] та еволюційних кібернетичних систем [18].

У даній роботі онтологічний контроль застосовується до *підсистеми пам'яті* автономного агента: формальна структура визначає, який контекст подається оператору при ініціалізації сесії, як оновлюється контекст простоючих місій, та як вимірюється якість витягування через сигнал корекції.

1.3 Постановка задачі

Мета: розробити архітектуру підсистеми пам'яті для автономних агентних систем, що забезпечує:

1. збереження контексту рішень між сесіями операторів;
2. автоматичне оновлення контексту для простоючих задач;
3. генерацію сигналу корекції для вдосконалення системи навчання.

Об'єкт дослідження: процеси людино-машинної взаємодії в автономних агентних системах з довгостроковими місіями.

Предмет дослідження: методи та моделі забезпечення безперервності контексту управління при ротації операторів.

2 Аналіз існуючих підходів

2.1 Системи пам'яті для діалогових агентів

MemGPT [11] реалізує віртуальне управління контекстом за аналогією з ієрархічною пам'яттю операційних систем. Generative agents [19] використовують епізодичну пам'ять для симульованих агентів. Mem0 [1] забезпечує довгострокову пам'ять для LLM через зовнішнє сховище.

A-MEM [22] розвиває парадигму епізодичної пам'яті у напрямку структурованого знання: кожне спостереження перетворюється на атомарну нотатку за принципом Zettelkasten з явними зв'язками між нотатками, утворюючи граф знань поверх потоку спостережень. Це наближає одиницю пам'яті до структурованого запису, проте зв'язки залишаються асоціативними, а не

каузальними — нотатка не фіксує відхилені альтернативи та не прив’язується до конституційного принципу.

Letta [12], комерціалізований наступник MemGPT, вводить два механізми, безпосередньо релевантні цій роботі. По-перше, *контекстні репозиторії* (context repositories) [6] — персистентні версіоновані сховища структурованого контексту, до яких агент звертається між сесіями; вони замінюють плоску архівну модель MemGPT типізованими колекціями з підтримкою запитів. По-друге, *контекстна конституція* (context constitution) [5] — декларативна специфікація того, до якого контексту агент має доступ і з яким пріоритетом, аналогічна політиці витягування, вираженій як конфігурація, а не як код. Ці механізми наближають пам’ять агента до витягування за провенансом рішень: контекстні репозиторії надають субстрат зберігання, а контекстна конституція — рудиментарну політику витягування. Проте одиницею витягування в Letta залишається контекстний блок (текстовий фрагмент з метаданими), а не оперативне рішення з провенансом, альтернативами та конституційним обґрунтуванням.

Систематичний огляд механізмів пам’яті для LLM-агентів подано у Zhang et al. [25].

Обмеження: усі розглянуті системи — MemGPT, Generative Agents, A-MEM, Mem0, Letta — використовують як одиницю витягування діалоговий обмін, спостереження персонажа або контекстний блок, а не оперативне рішення з повним провенансом.

2.2 Витягування для програмних агентів

RepoCoder [24] та подібні системи витягують фрагменти коду за семантичною подібністю. CodeRAG-Bench [21] забезпечує систематичний бенчмарк для витягування з доповненою генерацією коду, охоплюючи документацію, довідники API та приклади коду як джерела витягування. SWE-bench [4] та SWE-agent [23] оцінюють агентів на реальних задачах з GitHub (issues), демонструючи, що витягування з урахуванням структури репозиторію та контексту задачі підвищує частку успішних розв’язків.

Обмеження: усі зазначені системи витягують *текст коду та ідентифікатори* — без провенансу рішень: чому цей код написано саме так, які альтернативи розглядались, які обмеження враховано.

2.3 Записи архітектурних рішень

Практика формального документування архітектурних рішень має тривалу історію в програмній інженерії. Формат Architecture Decision Record (ADR), запропонований Найгардом [9], визначає легковагий шаблон у форматі Markdown — назва, статус, контекст, рішення, наслідки — що зберігається безпосередньо разом із кодовою базою. Стандарт ISO/IEC/IEEE 42010 [3] формалізує опис архітектури на організаційному рівні, надаючи нормативну рамку для документування архітектурних точок зору та відповідностей. Y-формат Цьорнера [27] структурує рішення у вигляді шаблонного висловлювання: «В контексті [ситуація], зіткнувшись із [проблемою], ми вирішили [варіант], щоб досягти [якості], приймаючи [компроміс]». Zimmermann et al. [26] розширює практику управління ADR до міжпроектного керівництва, додаючи моделювання простору проблем та систематичне відстеження залежностей між рішеннями.

ADR фіксують *провенанс рішень* — саме той контент, що відсутній у витягуванні code-RAG. Проте механізм витягування ADR залишається файловим: рішення знаходять переглядом директорії за назвою та датою, а не семантичним запитом. Відсутні індекс вбудовувань (embedding index), гібридне витягування, перерангування за релевантністю до поточної задачі. Крім того, ведення ADR потребує ручного авторства: у високошвидкісному робочому процесі з продуктивністю 14,7 PR/день кураторське навантаження на підтримку повного реєстру ADR стає

заборонно високим.

За військовою аналогією, ADR подібні до журналу рішень командира — цінний артефакт, але пошук у ньому здійснюється за датою та підрозділом, а не за семантичною релевантністю до поточної оперативної ситуації.

2.4 Моделі з довгим контекстом

Gemini 1.5 Pro [20] підтримує контекст до 1M+ токенів. Проте дослідження [2, 7, 8] демонструють деградацію уваги після ~200K токенів: модель “губить” інформацію в середині довгого контексту. Плоске завантаження всього контексту місії не масштабується з тривалістю місії.

2.5 Системи управління знаннями в ситуаційних центрах

Онтологічно-керовані системи підтримки рішень [13] забезпечують формальне представлення знань для ситуаційної обізнаності. Palagin et al. [17] демонструють ефективність інтеграції нейромережових та онтолінгвістичних парадигм. Проте жодна з існуючих систем не поєднує онтологічну структуру з дворежимним витягуванням для забезпечення безперервності управління при ротації операторів.

Синтез. Жодна з п’яти ліній робіт — діалогова пам’ять, витягування для програмних агентів, реєстри архітектурних рішень, моделі з довгим контекстом, онтологічно-керовані системи — не розглядає провенанс рішення як першокласну одиницю пам’яті із семантичним витягуванням. Жодна не забезпечує примітиву повільного циклу оновлення (*slow-loop refresh*), що підтримує актуальність пам’яті про неактивні задачі на горизонтах у кілька тижнів — інтервалі, характерному для ротації операторів у штабних процесах. Архітектура, описана в цій роботі, займає перетин цих ліній: вона запозичує структуру провенансу з ADR, семантичне витягування з Code-RAG, персистентність із систем діалогової пам’яті та селективне завантаження контексту з RAG — додаючи дворежимне витягування (*dual-mode retrieval*) як нову архітектурну примітиву, спеціально спроектовану для забезпечення безперервності управління при зміні операторів.

3 Формалізація проблеми

3.1 Три характеристики довгострокових місій

Довгострокова автономна місія характеризується трьома вимірюваними властивостями, верифікованими на пілотному датасеті [10]:

Горизонт. Пов’язані рішення охоплюють дні — тижні. У верифікаційному розгортанні: 105 днів активної роботи з медіанним інтервалом між комітами менше 2 годин, але архітектурними потоками тривалістю 4–6 тижнів.

Композиційність. Рішення *A* тижня 2 обмежує рішення *B* тижня 6. Вибір схеми бази даних визначив контракти 29 інструментів, побудованих впродовж наступних п’яти тижнів.

Персистентність. Стан місії є спільним між сесіями операторів. Агент не зберігає історію діалогу — він повертається до зміненого стану місії кожної нової сесії.

3.2 Проблема ініціалізації сесії

Кожна нова сесія оператора починається із завантаження контексту. Вимірювання на 304 сесіях: медіана вартості ініціалізації — 30 115 вхідних токенів, з яких ~59% — надлишковий контекст.

Медіана коефіцієнта надлишковості (файли, зчитані при ініціалізації, але жодного разу не використані під час сесії) — 60%.

При ротації операторів проблема загострюється: новий оператор не має *жодного* контексту рішень попередника — лише стан місії без обґрунтувань.

3.3 Формальна постановка

Нехай T — кількість токенів ініціалізації, K — кількість зчитувань файлів/записів, W — коефіцієнт надлишковості.

- **Pull-режим** зменшує K та T при ініціалізації: замість плоского завантаження — цільове витягування за описом задачі.
- **Push-режим** підтримує актуальність контексту: для простоюючих задач/секторів фонове оновлення забезпечує, що при відновленні роботи контекст вже актуальний.
- **Метрика якості**: частка корекцій оператора, спричинених недостатнім контекстом (retrieval-correction signal).

Цільові показники: $T' \leq 10\,000$ токенів, $W' \leq 20\%$.

4 Архітектура трирівневої пам'яті

Підсистема пам'яті декомпонується на три рівні з різною семантикою витягування та субстратом зберігання. Рисунок 1 показує загальну архітектуру з обома режимами витягування.

4.1 Предметний рівень (Domain Layer)

Предметний рівень містить оперативні дані домену.

Табл. 1: Зіставлення предметного рівня: юридичний AI та управління БПЛА.

Юридичний AI (верифікація)	Управління БПЛА (застосування)
Закони, судові рішення, регламенти	Оперативна обстановка, картографія, розвіддані
Законодавчі зміни, судова практика	Зміни обстановки, нові загрози
70+ MCP-інструментів пошуку	Сенсори, системи спостереження
Семантичне витягування з метаданими (юрисдикція, дата, тип)	Семантичне витягування з метаданими (сектор, тип загрози, час)

Витягування: семантична подібність (cosine similarity) з опціональною фільтрацією за метаданими. Архітектурна зміна порівняно з прямими викликами інструментів: маршрутизація через єдину підсистему пам'яті забезпечує уніфіковане логування всіх запитів, вимірювання частоти промахів та єдиний формат відповіді.

4.2 Робочий рівень (Workflow Layer)

Робочий рівень містить контекст рішень — провенанс, відсутній як у вихідному коді, так і в предметних даних.

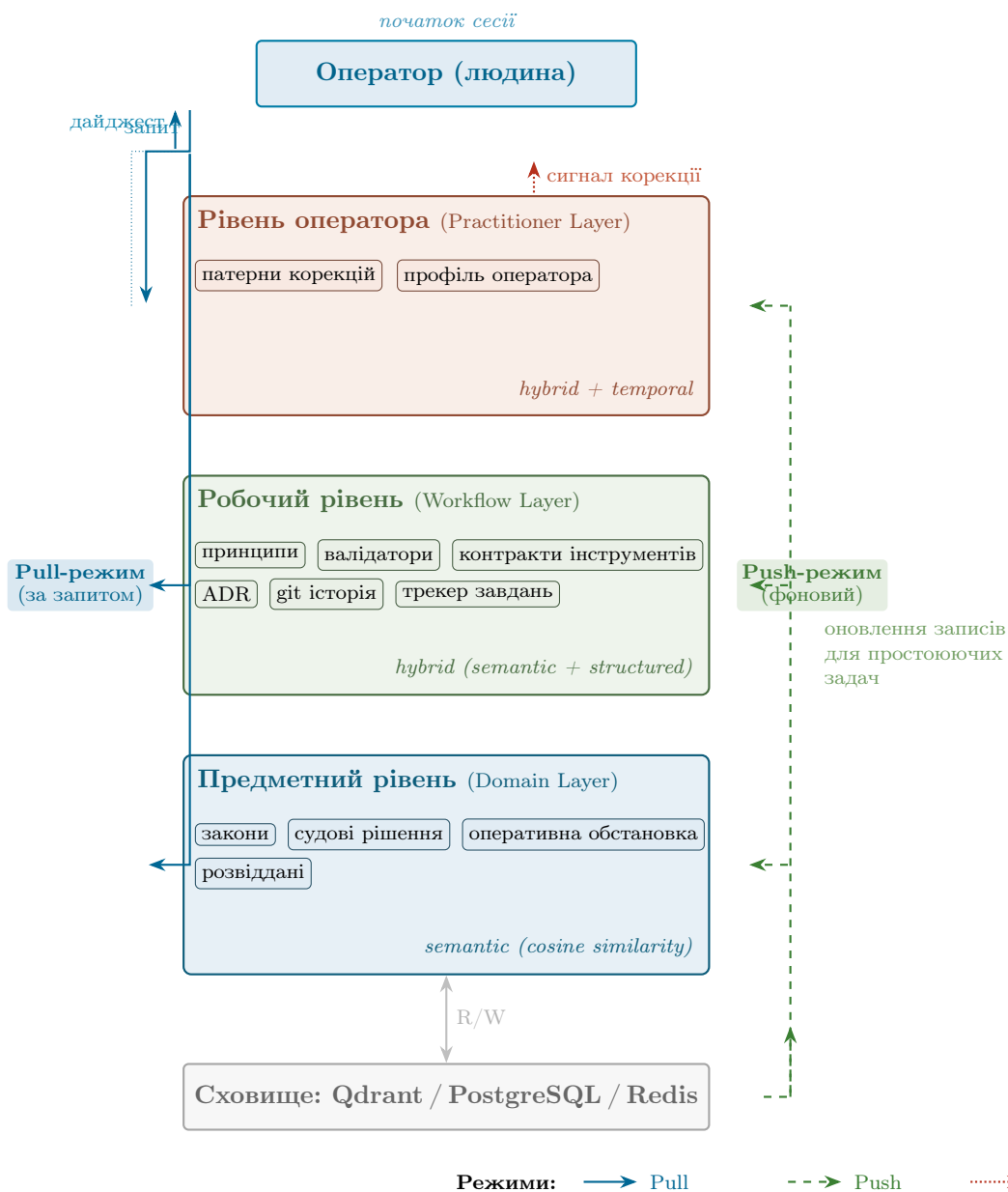


Рис. 1: Трирівнева архітектура персистентної пам'яті з дворежимним витягуванням. Pull-режим (суцільна лінія) ініціюється запитом оператора на початку сесії та паралельно звертається до всіх трьох рівнів. Push-режим (пунктирна лінія) фоново оновлює контекст простоючих задач. Сигнал корекції (точкова лінія) повертається від рівня оператора при виявленні пропущеного релевантного контексту.

Табл. 2: Шість джерел робочого рівня та їхні військові аналоги.

#	Оригінал	Військовий аналог
1	Реєстр принципів (порушення, що були скориговані)	Журнал порушень ROE та виправлених помилок
2	Визначення валідаторів	Правила перевірки дотримання ROE/SOP
3	Контракти інструментів (70+)	Контракти систем озброєння та сенсорів
4	Записи архітектурних рішень (ADR)	Рішення командира з обґрунтуванням
5	Історія git з семантичним збагаченням	Хронологія дій місії
6	Стан трекера завдань	Стан завдань/цілей місії

Витягування: гібридне — семантична подібність + структуровані фільтри (компонент, тип місії, часовий діапазон).

4.3 Рівень оператора (Practitioner Layer)

Рівень оператора містить патерни рішень конкретного оператора, витягнуті з pipeline корекцій [10].

Ключова властивість: цей рівень ніколи не зберігає сирий контент — тільки структуровані абстракції: клас корекції, семантична категорія, афектований компонент, результат (прийнято/відхилено/модифіковано) та однореченнєве резюме.

Застосування при ротації: новий оператор отримує профіль попередника — типові корекції, зони підвищеної уваги, рішення місії з обґрунтуваннями — у компактному дайджесті при ініціалізації сесії.

Витягування: гібридне з фільтрацією за часом та результатом.

Фаза 0: міст промпт–коміт. Перед розгортанням повноцінного рівня оператора встановлюється мінімальний колектор даних. Хук `UserPromptSubmit` у Claude Code створює orphan-коміт у bare-репозиторії git для кожного промпту оператора. Кожний запис містить: текст промпту, мітку часу, ідентифікатор сесії, репозиторій, гілку та режим дозволів.

Протягом 4–6 тижнів пасивного збору формується природномовний корпус, що забезпечує вхідні дані для трьох задач:

- розмежувальні мітки (disambiguation labels) — класифікація типу наміру оператора;
- розподіли частот тем (topic-frequency distributions) — виявлення домінантних робочих контекстів;
- патерни перемикання між проектами (cross-project switching patterns) — моделювання переходів оператора між репозиторіями та завданнями.

Цей корпус є сировинним матеріалом для embedding-pipeline рівня оператора: з нього витягуються структуровані абстракції, описані вище, без збереження сирого контенту промптів у довготривалій пам’яті.

Продуктивність: затримка захоплення становить 15 мс на один промпт — непомітна в межах 5-секундного таймауту хуків Claude Code. Фаза 0 розгорнута з 9 травня 2026 р.

Військова аналогія. Фаза 0 аналогічна пасивному збору радіоелектронної розвідки (SIGINT): накопичення даних без втручання в оперативну діяльність, формування базового розуміння патернів поведінки оператора до розгортання активних систем аналізу та адаптації.

5 Дворежимне витягування

5.1 Pull-режим (активні сесії)

При вході нового оператора система виконує запит до всіх трьох рівнів паралельно. Опис поточної задачі/сектора вбудовується у вектор (embedding), зіставляється з проіндексованим сховищем пам'яті, і k найрелевантніших записів збираються у компактний дайджест. Оператор починає роботу з фокусованим контекстом ($K' \ll K$, $T' \ll T$) і може витягувати додаткові записи на вимогу.

5.2 Push-режим (фонове оновлення)

Для простоюючих задач та секторів місії: планований фоновий процес відстежує активність і оновлює записи пам'яті пропорційно до швидкості релевантних змін. При відновленні роботи в секторі pull-запит повертає *актуальний* контекст без потреби повторного збору інформації.

Частота оновлення визначається як функція трьох сигналів активності:

$$f_{\text{refresh}}(\text{task}) = g(\text{commits-touching-task}, \text{comments-on-issue}, \text{time-since-last-pull}) \quad (1)$$

де g — монотонно зростаюча функція за кожним аргументом, обмежена зверху одним оновленням на 24 години та знизу — одним оновленням на 7 днів для будь-якої задачі з позначкою LONG-TERM. Таке обмеження запобігає як надмірному витрачання обчислювальних ресурсів на високоактивних задачах, так і повному застаріванню контексту для задач з низькою активністю.

Кожне оновлення породжує три артефакти:

- **Виконавче резюме змін** — генерується мовною моделлю на основі дифів релевантних комітів та коментарів до задачі; містить стислий виклад усіх суттєвих подій від попереднього оновлення.
- **Дельта лінеажу інструментів** — перелік нових, модифікованих та виведених з експлуатації інструментів із дифами параметричних схем, що дозволяє оператору при поверненні одразу бачити зміни в доступному арсеналі засобів.
- **Відкриті питання** — нерозв'язані альтернативи та точки прийняття рішень, позначені сумаризатором як такі, що потребують втручання оператора при відновленні роботи над задачею.

За військовою аналогією, цей механізм відповідає автоматизованій підготовці брифінгу передачі зміни: система формує зведення навіть тоді, коли жоден оператор не перебуває на бойовому чергуванні, — щоб наступна зміна при заступанні отримала актуальну обстановку без затримки на повторний збір інформації.

Критично для ротації: між змінами операторів push-режим оновлює контекст місії новими подіями, рішеннями інших секторів, змінами обстановки — так що наступна зміна починає з актуального стану.

5.3 Чому необхідні обидва режими

- **Pull без push:** після тривалої паузи контекст застарілий — оператор отримує рішення тижневої давності.

- **Push без pull:** надлишковий контекст при кожній сесії — оператор завалений інформацією з усіх секторів.
- **Комбінація:** push підтримує актуальність, pull фокусує витягування на поточну задачу.

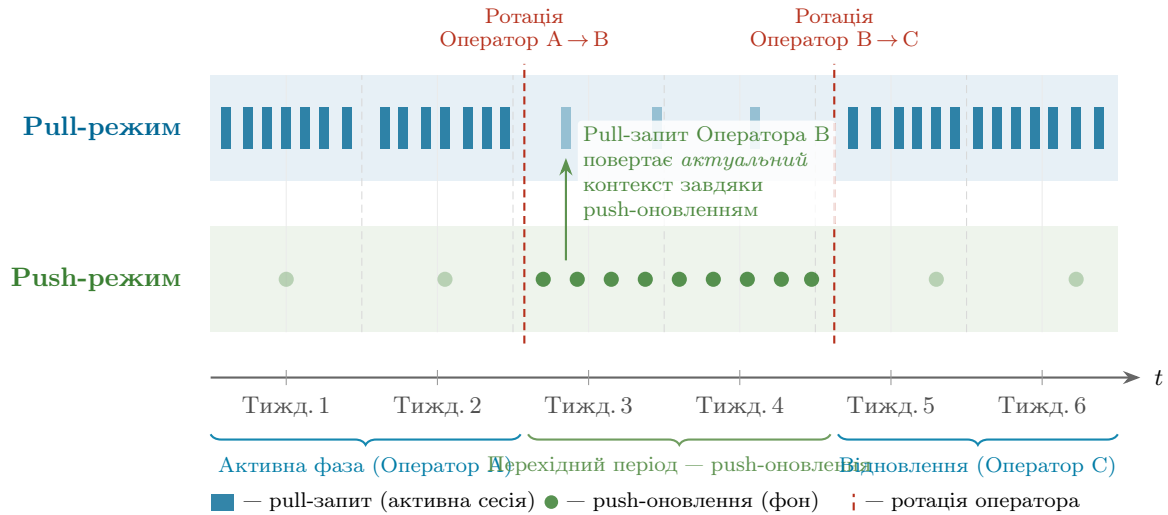


Рис. 2: Дворежимне витягування протягом шеститижневого циклу місії. У pull-режимі (верхня доріжка) кожна сесія оператора ініціює контекстний запит; у push-режимі (нижня доріжка) система періодично оновлює кешований контекст завдань. Під час ротації оператора (штрихові лінії) push-оновлення гарантують, що перший pull-запит нового оператора повертає актуальний стан місії без ручної реініціалізації.

5.4 Сигнал корекції витягування

Визначаємо *корекцію витягування* як корекцію оператора, яка була б непотрібною, якби підсистема пам'яті вчасно подала релевантний контекст.

Приклад: оператор БПЛА коригує маршрут, бо не знав про зміну обстановки в секторі. Якби push-режим оновив контекст місії, корекція не знадобилась би.

Цей сигнал виконує подвійну функцію:

1. **Метрика якості** підсистеми пам'яті: частка корекцій, спричинених витягуванням, вимірює ефективність push/pull-механізмів.
2. **Додатковий сигнал навчання:** кожна корекція витягування маркує конкретний промах системи — що мало бути подано і коли — створюючи датасет для вдосконалення моделі витягування.

Сигнал масштабується з автономністю агента: чим автономніший агент, тим більше рішень він приймає самостійно, і тим більше корекцій виникає через недостатній контекст.

6 Деталі реалізації

6.1 Інструмент запиту до пам'яті

Інтерфейсом витягування контексту для агента слугує єдиний MCP-інструмент `workflow_memory_query`, що забезпечує уніфікований доступ до всіх трьох рівнів пам'яті. Вибір єдиного інструменту замість окремих точок доступу до кожного рівня є принциповим проектним рішенням: агент

формулює один запит природною мовою, а підсистема пам'яті самостійно маршрутизує його до релевантних рівнів та агрегує відповіді.

Параметри інструменту:

- **task_description** (рядок, обов'язковий) — природномовний опис поточної задачі або оперативної ситуації.
- **scope** (перелічення: `all|domain|workflow|practitioner`) — рівні пам'яті для запиту. За замовчуванням: `all`.
- **token_budget** (ціле число, за замовчуванням 8000) — максимальна кількість токенів у зібраному дайджесті.
- **filters** (об'єкт, опціональний) — структуровані фільтри: компонент, рівень критичності, часовий діапазон.

Інструмент повертає *структурований прозовий дайджест* із такими складовими:

1. заголовки секцій відповідно до кожного рівня пам'яті, з якого витягнуто записи;
2. оцінка релевантності (relevance score) для кожного запису;
3. посилання на джерело (провенанс) — конкретний коміт, рішення командира, запис трекара або документ домену.

Формат дайджесту спроектовано для безпосередньої ін'єкції у контекстне вікно агента без додаткової постобробки. Токенний бюджет забезпечує контрольовану вартість ініціалізації: підсистема ранжує записи за релевантністю і відсікає нижні записи, коли сумарний обсяг перевищує бюджет.

Військова аналогія. У системах управління БПЛА або ситуаційних центрах еквівалентом є *уніфікований інтерфейс розвідувального запиту*: оператор формулює інформаційну потребу один раз, а система агрегує дані з усіх доступних джерел у єдину розвідувальну зведення. Уніфікований інтерфейс запиту є реалізацією принципу *інформаційного злиття* (information fusion), що є стандартним елементом архітектур C4ISR.

6.2 Оркестратор довгострокових задач

Push-режим реалізований як окремий Docker-контейнер з власним стоп-розкладом, ізольований від основного серверу обробки запитів. Архітектурна ізоляція забезпечує дві властивості: (1) фонові процеси оновлення не конкурують за ресурси з активними сесіями операторів; (2) збій оркестратора не впливає на доступність pull-режиму.

Цикл роботи оркестратора:

1. **Виявлення задач.** Оркестратор надсилає запит до трекара задач (Plane API) і отримує список задач із позначкою `LONG-TERM`.
2. **Обчислення пріоритету оновлення.** Для кожної задачі обчислюється пріоритет оновлення за формулою (1).
3. **Генерація виходів.** Для кожної задачі, що підлягає оновленню, оркестратор генерує три структуровані виходи за допомогою LLM-сумаризатора: резюме для командира, дельту лінійки інструментів та відкриті питання.
4. **Індексація та зберігання.** Згенеровані виходи вбудовуються у вектори та зберігаються у колекції Qdrant робочого рівня пам'яті з метаданною позначкою `source: push-refresh`.

Військова аналогія. Оркестратор є програмним аналогом *автоматизованої підготовки розвідувальних зведень між змінами*. Наступна зміна починає роботу не з порожнього аркуша, а з актуального зведення.

7 Експериментальна верифікація

7.1 Платформа верифікації

Архітектура верифікована на платформі юридичного штучного інтелекту LEX AI [10]: 70+ MCP-інструментів, 380M+ записів у pipeline даних, 1 547 об'єднаних pull-запитів за 105 днів роботи одного практика з LLM-агентом. Ця платформа є стрес-тестом підсистеми пам'яті: один оператор з максимальною пропускнуою здатністю генерує щільний потік рішень, що моделює навантаження автономної місії з обмеженим людським контролем.

7.2 Метрики

Чотири кількісні метрики операціоналізують критерії успішності архітектури.

Метрика	Метод вимірювання	Базове	Ціль
Вартість ініціалізації	Вхідні токени при першому API-виклику; $N=304$	30 115	$\leq 10K$
Коефіцієнт надлишковості	Файли зчитані, але не редаговані / всього; $N=180$	60%	$\leq 20\%$
Точність витягування принципів	Тестовий набір з edit-traces	н/д	$\geq 80\%$
Латентність push-оновлення	Час від коміту до дайджесту	н/д	≤ 24 год

Табл. 3: Метрики оцінювання з базовими значеннями та цільовими показниками.

П'ята метрика — *частота корекцій витягування* (Розділ 5.4) — має якісний характер на Фазі 1. Очікувана траєкторія — спадна крива протягом перших 6–8 тижнів розгортання.

7.3 Попередні результати

Вартість ініціалізації (Експеримент 1, $N=304$ сесій). Медіана вхідних токенів першого виклику 30 115 (середнє 29 693; $\sigma=7\,594$; P10/P90: 18 540/41 774). З цього обсягу $\sim 17\,600$ токенів (59%) витрачаються на створення кешу для CLAUDE.md та системного контексту — завантажуються безумовно для кожної сесії. Кількість зчитувань файлів в середньому 23,1 (медіана 14, P90: 61), проте 72% сесій мають нуль викликів Read у фазі ініціалізації.

Зростання CLAUDE.md (Експеримент 2, 25 комітів за 85 днів). Файл CLAUDE.md зріс із 4 099 до 24 148 символів (152 до 474 рядків). Лінійна регресія: 216,7 симв./день при $R^2 = 0,87$.

Коефіцієнт надлишковості (Експеримент 3, $N=180$ сесій). Медіана коефіцієнта надлишковості — 60% (середнє 56,7%; $\sigma=26,3\%$). 66% сесій витрачають більше половини зчитувань марно. Зчитування вихідного коду: 78% надлишковість; файли пам'яті: 66,5%. Ця метрика є консервативною проксі: файл може бути зчитаний для контексту без подальшого редагування.

Фаза 0 (міст промпт-коміт) розгорнута з 9 травня 2026 р.; розподільний аналіз потребує 4–6 тижнів збору.

7.4 Загрози валідності

Один проєкт, один практик. Архітектура верифікована в контексті одного проєкту та одного практика. Пом'якшення: архітектура є проєктно-агностичною. В контексті управління

БПЛА окремий оператор із закріпленим комплексом — типовий сценарій першої лінії.

Змішування з дозріванням кодової бази. Контроль: А/В-оцінювання шляхом увімкнення та вимкнення підсистеми пам'яті на тому самому розподілі задач.

Якість реєстру принципів. На Фазі 1 реєстр курується людиною; точність витягування обмежена якістю курації. Адресується на Фазі 2 напівавтоматичним витягуванням принципів з edit-traces.

7.5 Застосовність до управління БПЛА та ситуаційних центрів

Метрики платформи юридичного AI зіставляються з метриками бойових систем:

Табл. 4: Зіставлення метрик верифікації з бойовими метриками.

Метрика LEX AI	Бойовий аналог
Токени ініціалізації (T)	Час підготовки оператора до управління
Коефіцієнт надлишковості (W)	Когнітивне навантаження при зміні вахти
Retrieval-correction signal	Непотрібні корекції = потенційно запобігнуті інциденти
Час передачі контексту	Час передачі управління — метрика бойової готовності

8 Обговорення

8.1 Від пам'яті програмного агента до пам'яті бойової системи

Запропонована архітектура є доменно-агностичною: трирівнева декомпозиція та дворежимне витягування працюють однаково для юридичного AI та управління БПЛА. Предметний рівень заповнюється даними домену (законодавство або оперативна обстановка), робочий рівень — рішеннями місії, рівень оператора — профілем конкретної людини.

Ключова відмінність — вимоги до латентності: мілісекунди для систем реального часу (БПЛА) проти секунд для юридичного AI. Це впливає на архітектуру push-режиму: для БПЛА оновлення мають відбуватися в реальному часі, а не за розкладом.

8.2 Еволюційна кібернетика та адаптація пам'яті

Palagin et al. [18] запропонував рамку для аналізу систем, де цілі, обмеження та структури самі еволюціонують — на відміну від класичної теорії управління з фіксованою цільовою функцією.

Підсистема пам'яті місії є саме такою еволюційною системою: контент пам'яті, пріоритети витягування та навіть структура рівнів адаптуються до зміни обстановки. Push-режим є механізмом еволюційного оновлення: він не просто підтримує актуальність, а перебудовує контекст у відповідь на якісні зміни ситуації.

8.3 Зв'язок з онтологічно-керованими системами

Три рівні пам'яті природно зіставляються з концепціями онтологічно-керованих систем [13]:

- **Предметний рівень** = онтологія домену (формальне представлення знань про предметну область).
- **Робочий рівень** = онтологія процесу управління (рішення, їх обґрунтування, залежності).
- **Рівень оператора** = модель когнітивного профілю (патерни рішень, зони уваги).

Дворежимне витягування розширює цю аналогію: pull-режим відповідає активному запиту до онтології, push-режим — автоматичному оновленню онтології при зміні фактів про світ. Palagin et al. [17] демонструють, що інтеграція нейромережових та онтолінгвістичних підходів дає кращі результати, ніж кожна парадигма окремо — наша гібридна схема витягування (семантичне + структуроване) є реалізацією цього принципу.

8.4 Пам'ять як субстрат навчання з підкріпленням

Та сама інфраструктура, яка дозволяє одному практику виконувати сотні сесій протягом кількох місяців, *генерує* сигнал переваг, придатний для навчання з підкріпленням на основі зворотного зв'язку від людини (RLHF). Пам'ять і дані переваг є двома проєкціями одного і того ж довгострокового робочого процесу: кожна корекція після витягування є одночасно *збоєм підсистеми пам'яті та сигналом вирівнювання*.

Ця двоїстість не є випадковою: вона становить архітектурну тезу. Підсистема пам'яті не лише *споживає* дані вирівнювання — вона їх *продукує*. У бойовому контексті кожна корекція оператора під час управління БПЛА — зміна маршруту, перепризначення цілі — є одночасно записом пам'яті місії та зразком для навчання системи [10].

8.5 Пам'ять як поверхня масштабованого нагляду

Підсистема пам'яті виконує подвійну функцію: обслуговує агента під час виконання місії та генерує сигнал нагляду під час узгодження. Із зростанням автономності агента відношення спостережуваних результатів до внутрішніх рішень агента падає. Нагляд на рівні результатів стає вузькосмуговим каналом.

Підсистема пам'яті змінює цю динаміку. Кожне витягування є *спостережуваною подією* з відомим контекстним вікном, відомим результатом витягування та подальшим слідом редагування. Щільність сигналу на одиницю активності є вищою, ніж при RLHF на рівні результатів: кожна сесія генерує множину пар «витягнуто X , скориговано Y ».

8.6 Поведінка пам'яті при ескалації подій

Попередні підсекції розглядають підсистему пам'яті в режимі стаціонарного навантаження. Окремого аналізу потребує поведінка при *ескалації* — каскадному зростанні кількості одночасних подій, коли інформаційне навантаження на оператора та агента зростає експоненціально.

Каскадні сценарії. В бойовому контексті ескалація виникає при одночасному обстрілі кількох секторів, радіоелектронному придушенні каналів зв'язку та втраті контакту з частиною рою БПЛА. У програмному контексті — при каскадних виробничих інцидентах у кількох сервісах одночасно.

Деградований режим. Коли частота подій перевищує пропускну здатність обробки, підсистема пам'яті має переключитися з семантичного витягування на *тріаж на основі критичності*.

Формально, функція ранжування змінюється з

$$\text{score}(e) = \text{sim}(q, e) \longrightarrow \text{score}(e) = \alpha \cdot \text{criticality}(e) + (1 - \alpha) \cdot \text{sim}(q, e),$$

де $\alpha \rightarrow 1$ при зростанні частоти подій.

Ротація під час кризи. Найгірший сценарій для підсистеми пам'яті — ротація оператора посеред ескалації. Дайджест для нового оператора повинен передати не лише статичний стан, а й *динаміку*: що ескалює, що каскадує, які зв'язки між подіями виявлені, які рішення прийняті попереднім оператором. Це вимагає розширення формату дайджесту від плоского переліку фактів до темпоральної структури з причинно-наслідковими зв'язками.

Повна реалізація деградованого режиму виходить за межі поточної роботи (див. обмеження (iii) у секції 9), проте архітектурні точки розширення — функція ранжування з параметром критичності та черга пріоритетів у push-режимі — є частиною проекту.

8.7 Узагальнення на команди

Архітектура обслуговує одного практика. Багатооператорне розгортання породжує три виклики: *по-перше*, реєстр принципів стає багатозаписувачем з необхідністю вирішення конфліктів; *по-друге*, рівень оператора потребує індивідуальних представлень; *по-третє*, записи пам'яті можуть містити контекст, чутливий до рівня доступу.

Трирівнева декомпозиція спроектована для підтримки цих розширень: предметний рівень є спільним, робочий підтримує обмежені представлення через фільтрацію метаданих, а рівень оператора є за природою індивідуальним.

8.8 Чим це не є

Архітектура *не* є заміною моделей з довгим контекстним вікном — вона є витягувальним субстратом, що зменшує обсяг контексту, який повинна обробити модель [8, 20]. Це *не* є граф знань (не використовує RDF-трійки і не надає SPARQL-інтерфейс). Це *не* є корпоративна система управління знаннями (Enterprise KMS). Це витягувальний субстрат, оптимізований для одного операційного режиму — довгострокової агентної композиції малою кількістю практиків.

Військова аналогія. Це *не* є повноцінна система C4ISR. Це підсистема пам'яті, що інтегрується в існуючу інфраструктуру управління — компонент, що забезпечує безперервність інформаційного контексту для тих підсистем, які приймають рішення.

9 Обмеження

- (i) **Верифікація на одній платформі.** Архітектура верифікована на юридичному AI, не на БПЛА або ситуаційному центрі. Доменна агностичність — гіпотеза, що потребує перевірки.
- (ii) **Одиничний оператор.** Усі вимірювання отримані від одного практика. Поведінка при багатооператорній ротації потребує окремого дослідження.
- (iii) **Push-режим та деградований режим.** Фонове оновлення описане архітектурно, але повна реалізація ще не завершена. Тріаж на основі критичності (Розділ 8.6) є проєктним, не реалізованим.
- (iv) **Латентність.** Вимірювання латентності витягування отримані для не-реального часу. Для БПЛА потрібна оптимізація під мілісекундні вимоги.

10 Висновки

Запропоновано архітектуру підсистеми персистентної пам'яті для автономних агентних систем, що виконують довгострокові місії з ротацією операторів.

Основні результати:

1. **Трирівнева декомпозиція** (предметна область / робочий процес / оператор) забезпечує розділення контексту за семантикою витягування та терміном життя.
2. **Дворежимне витягування** (pull + push) вирішує проблему безперервності контексту: pull фокусує ініціалізацію на поточну задачу, push підтримує актуальність для простоючих задач та ротації операторів.
3. **Сигнал корекції витягування** надає метрику якості підсистеми пам'яті, яка масштабується з автономністю агента та слугує додатковим джерелом навчального сигналу.

Базові вимірювання на 304 сесіях підтверджують наявність проблеми: 60% надлишковість контексту при ініціалізації, що зростає лінійно з віком проєкту.

Архітектура є доменно-агностичною і застосовна до систем управління БПЛА та ситуаційних центрів, де ротація операторів є штатною процедурою, а безперервність контексту рішень — критичною вимогою бойової готовності. Окремо обговорено поведінку при ескалації подій: деградований режим із тріажем на основі критичності та проблему ротації посеред кризи.

Подальші дослідження: (1) верифікація на багатооператорній когорті; (2) реалізація push-режиму в реальному часі; (3) інтеграція з онтологічно-керованими системами підтримки рішень [13] для формалізації робочого рівня пам'яті; (4) реалізація деградованого режиму та емпірична верифікація тріажу на основі критичності.

Література

- [1] Prateek Chhikara, Deshraj Khare, Saquib Tariq, and Mudit Bansal. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- [2] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Joshua Ainslie, et al. RULER: What's the real context size of your long-context language models? *arXiv preprint arXiv:2404.06654*, 2024.
- [3] ISO/IEC/IEEE. ISO/IEC/IEEE 42010:2011 — systems and software engineering — architecture description, 2011.
- [4] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *Proceedings of the Twelfth International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06770.
- [5] Letta Team. Context constitution: Declarative retrieval policies for agent memory. <https://docs.letta.com>, 2026. Blog post / technical report.
- [6] Letta Team. Context repositories for persistent agent memory. <https://docs.letta.com>, 2026. Blog post / technical report.
- [7] Tianle Li, Junnan Ge, Jingbo Cheng, Siyuan Chen, Luyu Gao, et al. Long-context LLMs struggle with long in-context learning. *arXiv preprint arXiv:2404.02060*, 2024.
- [8] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *arXiv preprint arXiv:2307.03172*, 2023.

- [9] Michael Nygard. Documenting architecture decisions. <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>, 2011. Blog post.
- [10] Vladimir Ovcharov. Edit-trace oversight: Scalable alignment signal from agentic workflows. *arXiv preprint*, 2026. Under review.
- [11] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- [12] Charles Packer, Sarah Wooders, and Kevin Lin. Letta: The open-source framework for building stateful LLM agents (formerly MemGPT). <https://github.com/letta-ai/letta>, 2025. Software repository.
- [13] Alexander V. Palagin. Architecture of ontology-controlled computer systems. *Cybernetics and Systems Analysis*, 42(2):254–264, 2006. doi: 10.1007/s10559-006-0061-z.
- [14] Alexander V. Palagin, Serhiy L. Kryvyi, and Mykola G. Petrenko. On the automation of the process of extracting knowledge from natural language texts. *Natural and Artificial Intelligence International Book Series*, 2012.
- [15] Oleksandr Palagin, Vitalii Velychko, Kyrylo Malakhov, and Borys Shchurov. Distributional semantic modeling: A revised technique to train term/word vector space models applying the ontology-related approach. *arXiv preprint arXiv:2003.03350*, 2020.
- [16] Oleksandr Palagin, Vladislav Kaverinskiy, Anna Litvin, and Kyrylo Malakhov. OntoChatGPT information system: Ontology-driven structured prompts for ChatGPT meta-learning. *International Journal of Computing*, 22(2):170–183, 2023.
- [17] Oleksandr Palagin, Vladislav Kaverinskiy, and Kyrylo Malakhov. Fundamentals of the integrated use of neural network and ontolinguistic paradigms: A comprehensive approach. *Cybernetics and Systems Analysis*, 60:111–123, 2024. doi: 10.1007/s10559-024-00652-z.
- [18] Oleksandr Palagin, Dmytro Symonov, and Mykhailo Chervynskiy. Modelling evolutionary cybernetics: Ontology, invariants and design principles. *Information Technologies and Systems*, 6(6):3–29, 2025. doi: 10.15407/intechsys.2025.06.003.
- [19] Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. *arXiv preprint arXiv:2304.03442*, 2023.
- [20] Machel Reid, Nikolay Savinov, Denis Teber, et al. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530*, 2024.
- [21] Zora Zhiruo Wang, Akari Shi, Shuyan Yang, Ravi Anantha, Ryan A. Rossi, Neel Sundaresan Park, et al. CodeRAG-Bench: Can retrieval augment code generation? *arXiv preprint arXiv:2406.14497*, 2024.
- [22] Wujiang Xu, Zujie Liang, Jingquan Mei, Jiancheng Gao, Yongxin Qi, Feng Yang, Rui Yan, and Wei Liu. A-MEM: Agentic memory for LLM agents. *arXiv preprint arXiv:2501.13059*, 2024.
- [23] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.

- [24] Fengji Zhang, Bei Chen, Yue Zhang, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. RepoCoder: Repository-level code completion through iterative retrieval and generation. *arXiv preprint arXiv:2303.12570*, 2023.
- [25] Zeyu Zhang, Xiaohe Zhang, Yuanpei Wang, Shengjie Yan, Zhaokai Wang, Ge Yu, et al. A survey on the memory mechanism of large language model based agents. *arXiv preprint arXiv:2404.13501*, 2024.
- [26] Olaf Zimmermann, Lukas Wegmann, Heiko Koziolk, and Thomas Goldschmidt. Architectural decision guidance across projects: Problem space modeling, decision backlog management and cloud computing knowledge. In *Proceedings of the 12th Working IEEE/IFIP Conference on Software Architecture (WICSA)*, pages 85–94. IEEE, 2015. doi: 10.1109/WICSA.2015.23.
- [27] Stefan Zörner. *Softwarearchitekturen dokumentieren und kommunizieren: Entwürfe, Entscheidungen und Lösungen nachvollziehbar und wirkungsvoll festhalten*. Carl Hanser Verlag, München, 2 edition, 2015. ISBN 978-3-446-44361-9.